Global Solution 2026.2

Redes Neurais Artificiais, Deep Learning e Algoritmos Genéticos

Alerta Criadouro

Matheus Rebelo - RM 570260
Felipe Olmos - RM 570946
Luiz Guilherme - RM 573486

1. Problema e aplicação

Cerca de 75% dos focos do mosquito da dengue ficam dentro das casas (governo de SP, via Poder360, fev/2026). Em 2024 o Brasil teve 6,48 milhões de casos prováveis, o pior ano da história, e a cidade de São Paulo passou de 1,2 milhão de casos notificados. A larva vira mosquito em uns 10 dias, e a família quase sempre só percebe o problema quando alguém já ficou doente.

Público: famílias que moram em casas com quintal em bairros da cidade de São Paulo, usuárias do Alerta Criadouro, um aparelho com ESP32 que avisa quando o potinho sentinela do quintal está com água parada há dias.

Papel da rede neural: prever quantos casos de dengue a cidade vai ter em cada uma das próximas 4 semanas. Com isso o app mostra se a dengue está subindo, estável ou caindo, e os agentes de saúde e a prefeitura ganham um mês para se preparar. É uma tarefa diferente da de Machine Learning, que classifica se a próxima semana é de risco alto.

	Descrição
Entrada	As últimas 12 semanas da cidade: variação dos casos, Rt, temperatura média e mínima, umidade média e máxima e época do ano (8 atributos por semana)
Saída	Casos previstos para cada uma das 4 semanas seguintes
Tipo de tarefa	Regressão em série temporal

2. Base de dados

Fonte: InfoDengue (Fiocruz e FGV), que junta as notificações de dengue do SINAN com dados de clima. Usamos os dados semanais da cidade de São Paulo (código IBGE 3550308), de janeiro de 2015 a dezembro de 2025: 574 semanas e 30 colunas. Link: https://info.dengue.mat.br/api/alertcity?geocode=3550308&disease=dengue&format=csv&ew_start=1&ew_end=53&ey_start=2015&ey_end=2025

Por que é adequada: é oficial, real e longa (11 anos, com vários surtos, inclusive o recorde de 2024). É uma série semanal sem buracos, o formato de que uma rede recorrente precisa. E traz casos e clima na mesma semana, o que liga a previsão ao que o nosso aparelho mede no quintal. Antes do treino, o notebook confere o hash SHA-256 do arquivo (741ccfb2…97a03b), para garantir que a base não foi alterada.

Semana	**Casos**	**Rt**	**Temp. média (°C)**	**Umid. média (%)**	**Nível**
21/04/2024	76.880	0,96	21,12	78,79	4
05/05/2024	85.389	1,08	22,58	67,96	4
03/12/2023	931	1,16	23,16	83,30	3
24/12/2023	942	1,05	23,41	75,52	2

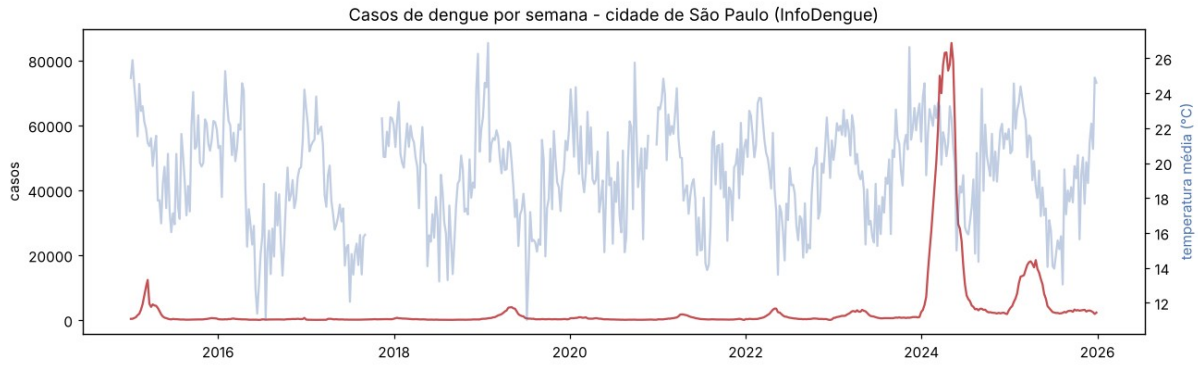


Figura 1 - Casos por semana (vermelho) e temperatura média (azul)

3. Preparação dos dados

- Valores faltando: 13 semanas sem temperatura média e umidades foram preenchidas por interpolação entre a semana anterior e a seguinte, o jeito certo numa série temporal.
- Log dos casos: os casos vão de 38 a 85 mil por semana. Usamos log(1 + casos) para a rede não ficar dominada pelos picos.
- Prever a variação, não o valor: na primeira tentativa a rede previa o número de casos direto e errou muito em 2024, porque nunca tinha visto valores tão altos. Mudamos a saída para quanto o log dos casos vai subir ou cair em relação à última semana conhecida. A dinâmica de um surto é parecida em anos grandes e pequenos, e isso a rede aprende.
- Janelas: cada exemplo tem 12 semanas de entrada (cerca de 3 meses) e 4 de saída. Formato final: 559 janelas × 12 semanas × 8 atributos.
- Divisão por tempo, sem embaralhar: treino 2015-2021 (350 janelas), validação 2022 (49) e teste 2023-2025 (154). O teste tem a epidemia de 2024, que a rede nunca viu.
- Padronização: StandardScaler ajustado só com o treino, para não vazar informação do futuro.

4. Arquitetura da rede

Escolhemos uma rede recorrente do tipo LSTM (Long Short-Term Memory). Os dados são uma sequência no tempo: o que acontece numa semana depende das anteriores. Uma MLP veria as 12 semanas como números soltos, sem ordem, e uma CNN é mais indicada para imagens. A LSTM tem uma memória interna controlada por portões (de esquecer, de entrada e de saída), que decidem o que guardar das semanas anteriores. Isso evita o problema da RNN simples, que esquece o começo de sequências longas. E o efeito do clima na dengue tem atraso: o calor de hoje vira mosquito em 1 a 2 semanas e caso notificado depois. A LSTM aprende esse atraso sozinha.

Como a rede processa os dados: ela lê as 12 semanas uma por uma, atualizando o estado interno com os 8 atributos de cada semana. No fim, esse estado resume a situação (por exemplo, "o Rt subiu três semanas seguidas e está quente"). Uma camada Dense com 4 neurônios transforma o resumo nas 4 variações previstas. Somando cada variação ao log dos casos da última semana e desfazendo o log, chegamos aos casos previstos.

Camada	Saída	Parâmetros
Entrada	12 semanas × 8 atributos	0
LSTM (32 neurônios na base; 64 na configuração final)	vetor de 32 (64)	5.248 (18.688)
Dropout	mesmo tamanho	0

Dense	4 valores (uma variação por semana)	132 (260)

5. Treinamento

Parâmetro	Valor
Função de perda	MSE (erro quadrático médio)
Otimizador	Adam
Learning rate	0,001 (0,0005 na configuração D)
Épocas	até 300 (com early stopping, parou entre 27 e 30)
Batch size	16 (32 na configuração E)
Semente	42, para dar para repetir

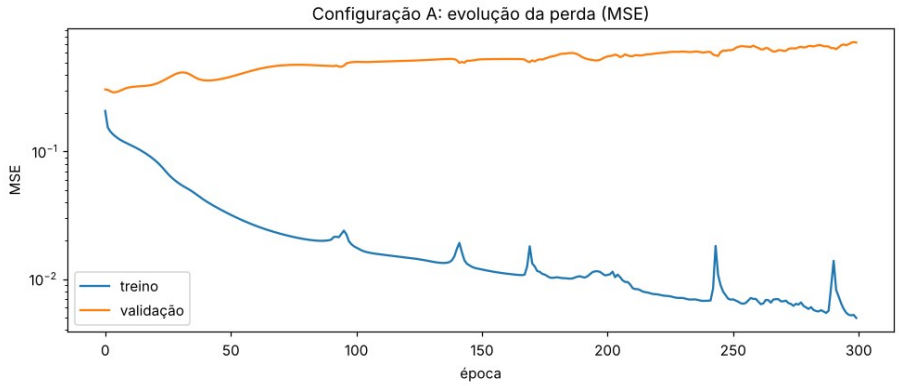


Figura 2 - Configuração A (sem regularização): perda de treino e validação

No primeiro treino (configuração A, sem regularização) a perda de treino caiu até 0,005, mas a de validação foi menor na época 4 (0,29) e depois subiu até 0,72. É overfitting: a rede decorou as semanas de treino.

6. Refinamento e otimização

Para corrigir o overfitting testamos dropout, early stopping (paciência de 25 épocas, voltando aos melhores pesos) e variações de neurônios, camadas, learning rate e batch size. A escolha foi pelo menor erro na validação (2022), sem olhar o teste.

Config.	Neurônios	Camadas	Dropout	LR	Batch	Épocas	RMSE log treino	RMSE log validação	MAE validação (casos)
A	32	1	-	0,001	16	300	0,071	0,846	479
B	32	1	0,2	0,001	16	28	0,378	0,547	279
C	64	1	0,3	0,001	16	27	0,375	0,531	274
D	32	2	0,2	0,0005	16	27	0,388	0,535	294
E	32	1	0,2	0,001	32	30	0,376	0,550	282
F	16	1	0,2	0,001	16	28	0,405	0,555	306
Ingênuo	-	-	-	-	-	-	-	0,561	349

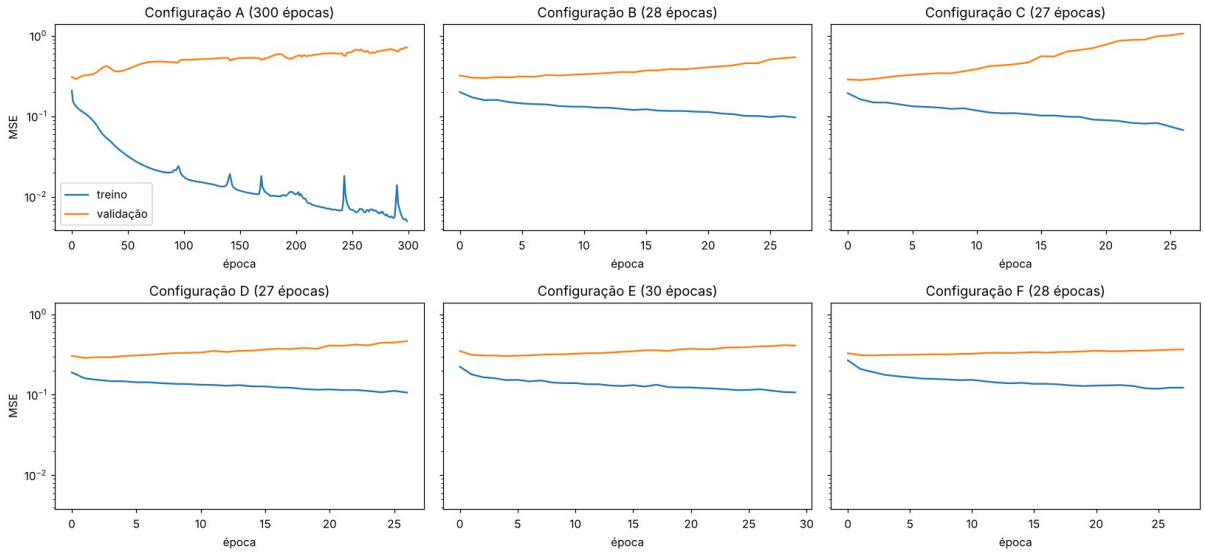


Figura 3 - Evolução da perda em cada configuração

O que aprendemos com a comparação:

- Dropout com early stopping resolveu o overfitting: o erro de validação caiu de 0,846 (A) para cerca de 0,53 a 0,55 em todas as outras.
- A configuração A tem o menor erro de treino e o maior de validação, a prova de que ela decorou.
- Mais neurônios com mais dropout (C) foi um pouco melhor que a base (B). Duas camadas (D) não ajudaram: com poucos dados, mais capacidade não compensa.
- Batch maior (E) e menos neurônios (F) deixaram a rede um pouco pior.
- Configuração final: C (64 neurônios, dropout 0,3, learning rate 0,001, batch 16, early stopping).

7. Avaliação no teste (2023-2025)

Modelo	RMSE (log)	RMSE (casos)	MAE (casos)
LSTM configuração C (final)	0,385	6.532	2.854
LSTM configuração A (sem regularização)	0,720	13.221	5.564
Ingênuo (repete a última semana)	0,454	8.134	3.357

Semanas à frente	MAE LSTM	MAE ingênuo	RMSE log LSTM	RMSE log ingênuo
1	1.475	1.570	0,20	0,22
2	2.541	2.784	0,33	0,36
3	3.538	3.950	0,43	0,50
4	3.863	5.125	0,51	0,63

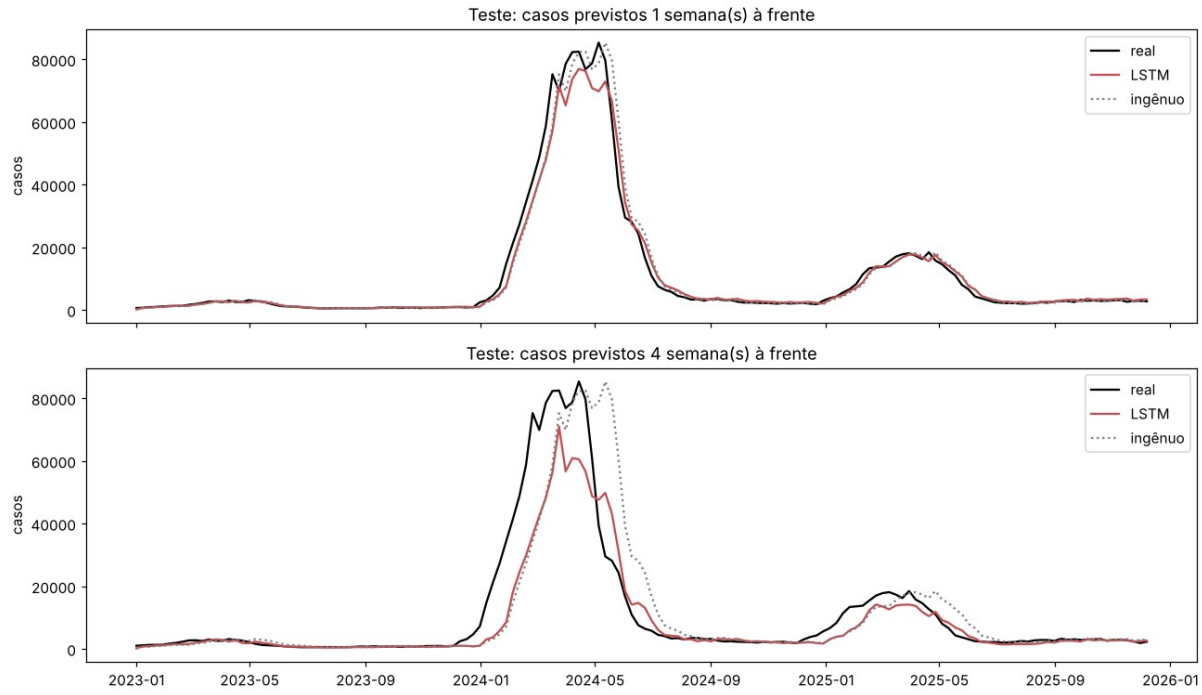


Figura 4 - Casos reais, previstos pela LSTM e pelo modelo ingênuo, 1 e 4 semanas à frente

Interpretação: o RMSE em log de 0,385 quer dizer errar, em geral, por um fator de cerca de 1,5 para cima ou para baixo. A LSTM erra menos que o modelo ingênuo em todos os horizontes, e a vantagem cresce com a distância: 4 semanas à frente o erro médio cai de 5,1 mil para 3,9 mil casos por semana (25% menor). A configuração sem regularização teve quase o dobro do erro, o que confirma que o refinamento melhorou a generalização.

- Pontos fortes: a rede aprendeu a dinâmica do surto e antecipa bem quando o pico passou e os casos vão cair.
- Erros: no começo de um surto ela atrasa a subida. Antes de o Rt disparar, as semanas de entrada parecem calmas e ela prevê estabilidade.
- Limitações: o erro em casos é grande nos picos (alguns milhares), porque 2024 foi muito maior que tudo no treino. Atrasos de notificação também atrapalham.
- É adequado? Sim, para mostrar a tendência e a ordem de grandeza das próximas semanas no app. Não serve para planejar leitos com precisão.

8. Demonstração com dados não usados no treino

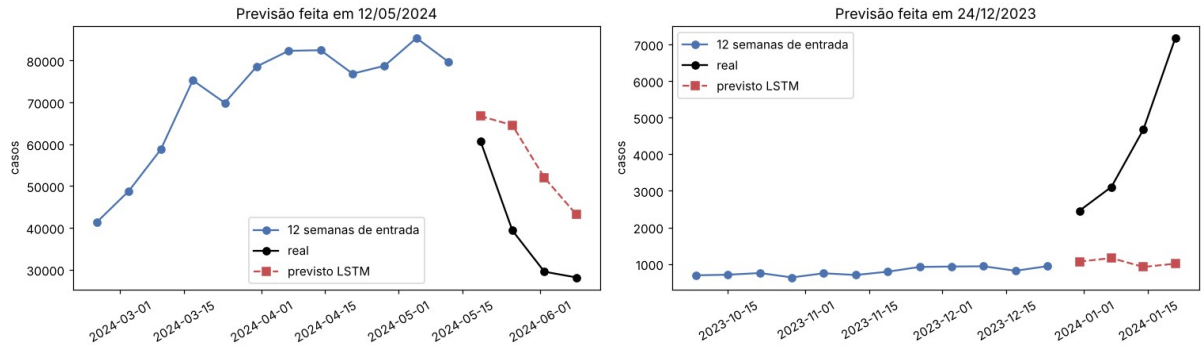


Figura 5 - Previsões feitas em 12/05/2024 e em 24/12/2023

Previsão feita em 12/05/2024 (79.706 casos na	Prevista	Real

semana)		
19/05/2024	66.734	60.783
26/05/2024	64.544	39.370
02/06/2024	52.011	29.522
09/06/2024	43.198	28.145

Previsão feita em 24/12/2023 (942 casos na semana)	Prevista	Real
31/12/2023	1.066	2.456
07/01/2024	1.163	3.107
14/01/2024	920	4.675
21/01/2024	1.009	7.176

Em maio de 2024, no fim do pico, a rede previu uma queda forte e acertou a direção; o modelo ingênuo diria que continuaria em quase 80 mil. Na véspera da epidemia (dezembro de 2023) a rede previu estabilidade e errou a subida. Por isso, no Alerta Criadouro, a LSTM não trabalha sozinha: o classificador de Machine Learning, que olha o Rt e a variação dos casos, avisou o risco alto já em 31/12/2023, e o aparelho no quintal avisa sempre que há água parada.

9. Integração com o Alerta Criadouro

A rede está ligada de verdade ao sistema. Criamos na API do projeto (a mesma API em Flask que recebe as leituras do ESP32) a rota GET /previsao, no arquivo api/previsao.py. Ela junta três coisas numa resposta só:

- a última leitura do quintal, gravada pelo ESP32 no CSV (OK, ATENÇÃO ou ALERTA);
- a probabilidade de risco alto na próxima semana, dada pelo modelo de Machine Learning (Random Forest);
- os casos previstos para as próximas 4 semanas, dados por esta rede LSTM, com a tendência (subindo, estável ou caindo).

Na primeira chamada a API treina os dois modelos com os mesmos dados do InfoDengue e os mesmos hiperparâmetros escolhidos nos notebooks (para a LSTM, a configuração C) e salva os arquivos na pasta api/modelos. As chamadas seguintes só carregam os modelos e respondem na hora. Conferimos que a rede da API dá a mesma previsão do notebook: para 12/05/2024, 66.733, 64.543, 52.011 e 43.198 casos, contra 66.734, 64.544, 52.011 e 43.198 no notebook (diferença de arredondamento). A rota também exige o token do aparelho e confere o hash da base antes de treinar.

Teste real feito no dia 06/11/2026, com a API rodando no Google Colab e acessada pela internet. A previsão é para as 4 semanas de janeiro de 2026, logo depois do último dado da base (28/12/2025):

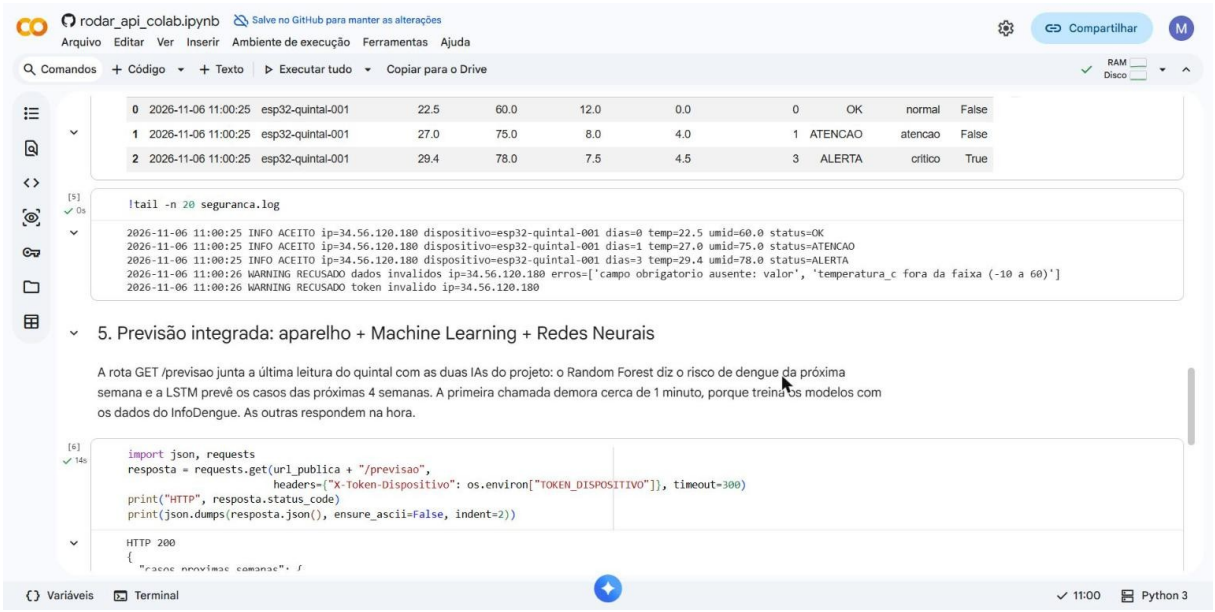


Figura 6 - Leituras do quintal gravadas e chamada da rota GET /previsao no Colab



Figura 7 - Resposta da API com a previsão da LSTM, o risco do Machine Learning e o status do quintal

```
HTTP 200
{
  "casos_proximas_semanas": {
    "casos_na_semana_base": 2306,
    "casos_previstos": [2645, 2680, 2289, 2448],
    "modelo": "LSTM (Redes Neurais)",
    "mudanca_em_4_semanas_pct": 6.2,
    "semanas": ["2026-01-04", "2026-01-11", "2026-01-18", "2026-01-25"],
    "tendencia": "estável"
  },
  "risco_proxima_semana": {"probabilidade_risco_alto": 0.28, "risco_alto": false},
  "status_quintal": "ALERTA",
  "mensagem_para_familia": "Risco de dengue em São Paulo na próxima semana: baixo.
     Casos nas próximas 4 semanas: tendência estável. Seu potinho está com água há dias:
     esvazie os vasos, baldes e pratinhos do quintal hoje."
}
```

Na prática, o app da família mostra essa mensagem: o aviso do quintal vem junto com o que a IA espera para a cidade. Quando a LSTM prevê casos subindo e o potinho está com água, a mensagem pede a vistoria no mesmo dia.

- Já funciona: a rede treinada, a rota GET /previsao na API, a previsão de 4 semanas com dados reais e a mensagem para a família, testadas pela internet.
- Evoluções futuras: baixar os dados novos do InfoDengue toda semana de forma automática, prever por bairro e usar a previsão do tempo como entrada.
- Acessibilidade e uso real: mensagem em linguagem simples, rede pequena (cerca de 19 mil parâmetros) que roda num servidor barato e responde em menos de 1 segundo depois de carregada.
- Segurança e dados: a rota pede o token, a rede usa só dados abertos da cidade inteira, sem dado pessoal das famílias, e a base é conferida por hash.

10. Arquivos

- redes_neurais/lstm_casos_dengue.ipynb: notebook completo, já executado, com todos os resultados.
- redes_neurais/lstm_casos_dengue.keras: rede final salva.
- api/previsao.py e api/app.py: integração da rede com a API (rota GET /previsao).
- dados/infodengue_sp_2015_2025.csv: base usada.
- Repositório: https://github.com/matheusrebelo04-maker/alerta-criadouro

Referências

- InfoDengue (Fiocruz e FGV). https://info.dengue.mat.br
- Hochreiter, S.; Schmidhuber, J. Long Short-Term Memory. Neural Computation, 1997.
- TensorFlow / Keras. Documentação oficial. https://www.tensorflow.org
- Poder360. 75% dos focos do mosquito da dengue estão dentro das casas, diz governo de SP. Fevereiro de 2026.